

1. Print all even numbers from 1 to 50 without arguments

```
def even_numbers():
```

```
    for i in range(1, 51):
```

```
        if i % 2 == 0:
```

```
            print(i)
```

```
even_numbers()
```

2. Sum of two numbers using arguments

```
def add(a, b):
```

```
    print("Sum =", a + b)
```

```
add(10, 20)
```

3. Return sum, average, minimum and maximum from a list

```
def list_details(lst):
```

```
    s = sum(lst)
```

```
    avg = s / len(lst)
```

```
    return s, avg, min(lst), max(lst)
```

```
print(list_details([10, 20, 30, 40, 50]))
```

4. Largest of three numbers using return

```
def largest(a, b, c):
```

```
    return max(a, b, c)
```

```
print("Largest =", largest(10, 50, 30))
```

5. Positional and keyword arguments together

```
def student(name, age, course):
```

```
    print("Name:", name)
```

```
    print("Age:", age)
```

```
    print("Course:", course)
```

```
student("Harini", age=20, course="Python")
```

6. Area of rectangle using positional arguments

```
def area(length, breadth):
```

```
print("Area =", length * breadth)
```

```
area(10, 5)
```

```
# 7. Simple Interest with default rate 5%
```

```
def simple_interest(p, t, r=5):
```

```
    si = (p * t * r) / 100
```

```
    print("Simple Interest =", si)
```

```
simple_interest(1000, 2)
```

```
# 8. Power of a number with default exponent 2
```

```
def power(num, exp=2):
```

```
    print("Result =", num ** exp)
```

```
power(5)
```

```
power(5, 3)
```

```
# 9. Sum using *args
```

```
def total(*args):  
  
    print("Sum =", sum(args))
```

```
total(10, 20, 30, 40)
```

10. Largest value using *args

```
def largest_value(*args):  
  
    print("Largest =", max(args))
```

```
largest_value(10, 50, 30, 80, 20)
```

11. Display all key-value pairs using **kwargs

```
def display(**kwargs):  
  
    for key, value in kwargs.items():  
  
        print(key, ":", value)
```

```
display(name="Harini", age=20, city="Salem")
```

12. Display employee details using **kwargs

```
def employee(**kwargs):  
  
    for key, value in kwargs.items():  
  
        print(key, ":", value)  
  
employee(id=101, name="Ravi", salary=25000)
```

13. Lambda function to find square

```
square = lambda x: x * x  
  
print(square(5))
```

14. Lambda function to find larger number

```
large = lambda a, b: a if a > b else b  
  
print(large(10, 20))
```

15. Convert Celsius to Fahrenheit using map()

```
celsius = [0, 20, 30, 40]  
  
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))  
  
print(fahrenheit)
```

16. Find squares using map()

```
nums = [1, 2, 3, 4, 5]
```

```
squares = list(map(lambda x: x**2, nums))
```

```
print(squares)
```

17. Extract even numbers using filter()

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
even = list(filter(lambda x: x % 2 == 0, nums))
```

```
print(even)
```

18. Extract numbers greater than 100 using filter()

```
nums = [50, 120, 80, 200, 150]
```

```
greater = list(filter(lambda x: x > 100, nums))
```

```
print(greater)
```

19. Recursive function to calculate power

```
def power_rec(a, b):
```

```
    if b == 0:
```

```
    return 1
```

```
    return a * power_rec(a, b - 1)
```

```
print(power_rec(2, 3))
```

```
# 20. Recursive Fibonacci series up to N terms
```

```
def fibonacci(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    return fibonacci(n-1) + fibonacci(n-2)
```

```
n = 6
```

```
for i in range(n):
```

```
    print(fibonacci(i), end=" ")
```